

From a Single Movie to a User Profile: A Reproducible Content-Based Movie Recommender

Student: Ivan Tsarikov

Assignment: A02 - Content-Based Filtering (Week 2)

Team: Individual

Date: 2026-09-22

Abstract - Content-based movie recommendation can fail when genre metadata is parsed incorrectly and a single active title is treated as the user's whole preference. This study repairs the MovieLens 100K baseline, averages three 19-dimensional genre vectors, ranks unseen films by cosine similarity, and uses Bayesian rating quality only for ties. Across 942 eligible users, item-to-item and profile-based Top-5 lists overlapped by 0.89 titles (17.9%), while cosine expanded profile coverage from 11.7% under raw overlap to 20.5%. The results support combining stable profiles with item-level retrieval and explicit novelty controls.

Index Terms - content-based recommendation, cosine similarity, user profiles, MovieLens, catalog discovery

1. Introduction

The original Week 2 application recommended movies from one selected title using broad genre labels. Its implementation also mapped 19 MovieLens genre flags onto only 18 names, shifting every known genre and discarding the final Western flag. The resulting interface could return semantically wrong or zero-overlap titles.

The practical goal was to preserve the starter application's simple web interface while making the model correct, explainable, and more representative of a user's taste. For example, a viewer who selected Toy Story, GoldenEye, and Four Rooms should receive an aggregate profile rather than a list determined by only one of those titles.

Contributions.

- Corrected all 19 genre positions, added strict parsing, deterministic ranking, watched-item exclusion, and executable tests.
- Replaced naive single-item matching with cosine similarity over an averaged three-movie profile and evaluated relevance, bias, and discovery behavior.

2. Related Work

Content-based systems represent items with attributes, construct a user-interest profile, and compare candidates with that profile [3]. Lops, de Gemmis, and Semeraro describe this profile-to-item matching pattern and identify limited content, overspecialization, and serendipity as central concerns [4]. MovieLens supplies a stable, documented benchmark for reproducible recommendation studies [1], [2].

Alternatives considered. Jaccard similarity was transparent but ignored preference frequency in the averaged profile. Raw dot product preserved weights but favored movies carrying many genre flags. Collaborative filtering could use rating interactions, but it would change the assignment from a content baseline and introduce cold-start behavior; ratings were therefore restricted to deterministic tie-breaking.

The implementation follows the web Fetch specification for asynchronous data loading [5] and otherwise uses no production libraries.

3. Method

Each movie is a binary vector over the 19 MovieLens genres in the documented order from Unknown through Western [2]. The profile is the component-wise mean of three distinct watched movies. Every unseen candidate receives cosine similarity to that profile; scores of zero are removed and the five highest scores are retained.

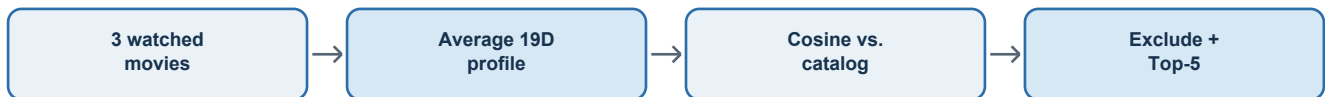


Figure 1. Production recommendation pipeline. Ratings never change the cosine score; they only resolve exact ties.

$$\text{profile} = (v1 + v2 + v3) / 3$$

$$\text{cosine}(P, C) = (P \text{ dot } C) / (\text{norm}(P) \times \text{norm}(C))$$

If either vector has zero magnitude, similarity is defined as 0. Candidates are ordered by cosine score, then Bayesian-smoothed rating quality, rating count, title, and movie ID. The quality tie-breaker is:

$$\text{quality}(i) = n_i / (n_i + 50) \times \text{mean}_i + 50 / (n_i + 50) \times \text{global_mean}$$

The prior weight of 50 prevents a movie with one unusually high rating from winning a tie over a well-supported title. It does not convert the method into collaborative filtering because rating data is not part of the content score.

3.1 Implementation pipeline

Stage	Operation	Correctness control
Load	Fetch u.item and u.data concurrently	Movie metadata is mandatory; ratings may fall back
Parse	Validate 24 item fields and 4 rating fields	Reject duplicates, NaN values, non-binary genres
Profile	Average three 19D vectors	Require three distinct known movie IDs
Retrieve	Score 1,679 eligible candidates	Exclude watched and zero-score movies
Rank	Bounded insertion into Top-5	Stable quality, count, title, and ID ties
Present	Write titles and scores with textContent	Preserve original layout and accessible status

3.2 Key design decisions

- Average instead of union: repeated genres receive weight 2/3 or 1 rather than being treated the same as a one-off genre.
- Cosine instead of raw overlap: vector-length normalization penalizes unrelated extra genres while retaining interpretable scores.
- Preserve presentation: only two selectors were added; the original 600-pixel card, controls, colors, and result box remain.

3.3 Configuration and environment

Top-k = 5; watched items = 3; positive cohort rating ≥ 4 ; Bayesian prior = 50 ratings; no random seed because ranking is deterministic. Execution environment: static HTML/CSS/JavaScript, macOS 26.4 on arm64; JavaScriptCore for behavior tests; Python 3.14.3 unittest for structural and dataset checks; Git revision a0ed77d.

4. Experiments

Dataset and protocol. MovieLens 100K contains 100,000 ratings from 943 users on 1,682 movies [1], [2]. For cohort evaluation, 942 users had at least three ratings ≥ 4 . Each user's earliest three positive ratings by timestamp formed the profile; the third was the active item for the item-to-item baseline. Both approaches excluded all three watched titles and returned Top-5. The 337 most-rated movies (top 20% of the catalog) were labeled head; the other 1,345 were long tail.

4.1 Worked Top-5 comparison

The illustrative history is Toy Story, GoldenEye, and Four Rooms. The single-item column deliberately uses Toy Story as the active item; the profile column uses all three movies.

Rank	Single active item	Cos.	Profile-based	Cos.
1	Aladdin and the King of Thieves	1.000	Close Shave, A	0.770
2	Aladdin	0.866	Rock, The	0.770
3	Goofy Movie, A	0.866	Clear and Present Danger	0.770
4	Wrong Trousers, The	0.816	Con Air	0.770
5	Grand Day Out, A	0.816	Firestorm	0.770

Table 1. Top-5 results generated by the production ranking code. The lists have no common title because one captures a narrow animation/comedy intent while the other emphasizes the repeated Thriller weight.

4.2 Manual verification

The three-movie profile has weights $1/3$ for Action, Adventure, Animation, Children's, and Comedy, plus $2/3$ for Thriller. Its squared magnitude is $5 \times (1/3)^2 + (2/3)^2 = 1$. Close Shave, A has Animation, Comedy, and Thriller, so the dot product is $1/3 + 1/3 + 2/3 = 4/3$ and its magnitude is $\sqrt{3}$. Therefore cosine = $(4/3) / \sqrt{3} = 0.7698$, which rounds to the application's 0.770.

4.3 Cohort results

Metric across 4,710 slots	Item-to-item	Profile cosine	Profile raw dot
Mean Top-5 overlap	0.89 titles	0.89 titles	n/a
Long-tail share	36.4%	35.1%	33.5%
Unique items exposed	367	344	196
Catalog coverage	21.8%	20.5%	11.7%
Mean rating count	155.5	162.3	184.1
Top-10 concentration	21.8%	21.6%	42.4%
Mean genres / result	2.25	2.89	3.87

Table 2. Deterministic re-run on the checked-in data and ranking policy. Item and profile overlap equals 17.9% of a five-item list.

4.4 Verification

A clean re-run produced 54 JavaScript assertions and 13 Python tests with no failures. Checks cover all 19 genre positions, Toy Story and Western fixtures, malformed data, repeat-safe parsing, profile averaging, zero-vector cosine, watched-item exclusion, Top-5 size, rating ties, DOM accessibility, and dataset integrity. The evaluator separately confirmed 1,682 movies, 100,000 ratings, 943 users, and global mean rating 3.52986.

5. Discussion

5.1 Item-to-item vs. profile-based

The representations serve different product surfaces. Item-to-item is a local intent query suitable for 'More like this'; the profile is a steadier home-page signal. Their average overlap was only 0.89 of 5 titles, so profile aggregation materially changes retrieval rather than merely reranking the same candidates. A production system should retrieve from both and rerank their union.

Decision dimension	Single active item	Three-movie profile
Intent	Immediate and narrow	Persistent and multi-interest
Noise sensitivity	High if active item is atypical	Lower, but averaging can blur tastes
Offline long-tail share	36.4%	35.1%
Offline catalog coverage	21.8%	20.5%
Recommended surface	Detail page / current session	Home page / return visit

5.2 Bias mitigation through cosine normalization

Raw dot product rewards every matching flag without penalizing unrelated flags. For a one-genre query, an exact one-genre candidate scores 1. A five-genre candidate with only that one match also has raw overlap 1, but cosine falls to $1/\sqrt{5} = 0.447$ because the four extra genres enlarge its norm. Thus a broad movie must match proportionally more of the profile to keep a high score.

Profile ranking metric	Raw dot	Cosine	Change
Mean genres per result	3.87	2.89	-25.3%
Top-10 concentration	42.4%	21.6%	-20.8 pp
Mean rating count	184.1	162.3	-11.8%
Long-tail share	33.5%	35.1%	+1.6 pp
Unique items exposed	196	344	+148
Catalog coverage	11.7%	20.5%	+8.8 pp

Cosine therefore mitigates genre-cardinality bias and nearly halves exposure concentration. It does not guarantee popularity neutrality: a broad item can genuinely align, and the rating tie-breaker still favors well-supported titles.

5.3 Catalog discovery and retention

Item-to-item uncovered slightly more of the long tail: 36.4% vs. 35.1%, with 367 vs. 344 unique titles. It produced more long-tail items for 356 users; the profile did so for 315, while 271 tied. The likely mechanism is centroid bias: averaging different interests moves the query toward common bridge genres, and the quality tie-breaker reinforces established movies.

The offline result supports a blended Top-5: retrieve profile candidates for stable relevance, add candidates from the most recent liked item, remove watched and near-duplicate titles, then enforce a bounded novelty rule such as at least two long-tail items above a minimum cosine threshold. An online test should use qualified play rate as the primary outcome and completion, negative feedback, repeat exposure, Top-10 concentration, and short-term retention as guardrails. The offline data does not establish that long-tail exposure causes retention.

Scope limits. MovieLens 100K is a historical explicit-rating dataset; rating count is only a popularity proxy. The 20/80 boundary is analytical, the three earliest positive ratings may not match real selection behavior, and no online fatigue or retention signal is available.

5. Discussion (continued)

5.4 Failure case, root cause, fix, and verification

Observed failure	Specific root cause	Correction and confirmation
Toy Story parsed as Children's, Comedy, Crime; Western-only movies had no genre	The parser sliced 19 flags but indexed them with an 18-name list that omitted Unknown	Added the complete ordered schema; tests now assert Toy Story = Animation/Children's/Comedy and a Western-only fixture = Western
An AI iteration replaced the starter interface with a new landing-page design	The implementation interpreted 'improve the app' as permission to redesign presentation	Compared against the original source, restored its HTML/CSS contract, added only two selectors, and cache-busted deployed assets

The second failure is important AI-management evidence: code correctness did not imply task correctness. Visual comparison against the stated reference was required to detect scope drift.

5.5 What worked

- Pure parsing and ranking functions made correctness testable without a browser or network.
- Cosine preserved interpretability while reducing broad-item concentration and increasing coverage over raw overlap.
- The rating-data fallback kept genre recommendation usable when u.data failed.

5.6 What surprised me or did not work

- Profile averaging did not improve long-tail discovery; item-to-item led every aggregate discovery measure.
- A rating tie-breaker improved deterministic quality but reintroduced some popularity pressure.
- The first UI revision was technically polished but violated the requirement to preserve the original application.

5.7 Next improvement

The next iteration would retrieve candidates from both the profile and latest liked item, then rerank with a capped novelty bonus. I would evaluate it offline for coverage and concentration, then online for qualified play and retention rather than claiming a causal retention gain from historical ratings.

6. AI Usage Disclosure

Tool. OpenAI Codex, GPT-5 family (exact service build not exposed in the session UI). Categories: source audit, code generation, debugging, test design, quantitative analysis, documentation, deployment checks, and report/PDF generation.

How it was used. Codex inspected the starter, recorded defects, implemented parsing and ranking changes, wrote tests, generated a cohort evaluator, checked source metadata, restored visual fidelity after feedback, and drafted this report. The unmodified attached session.json is the audit trail.

Personally verified. I compared the deployed interface against the original Week 2 source and rejected an AI-generated redesign; that intervention caused the visual restoration documented above. The code suite was re-run from a clean command during the session, and the report gives an independently reproducible line-by-line cosine check.

Not independently verified. Codex executed the full 942-user aggregation and reference lookup. I did not manually recompute all 4,710 recommendation slots; instead, the evaluator, raw files, exact protocol, test outputs, and session log are retained so the reported aggregates can be reproduced. No benchmark or causal retention claim is presented as personally verified.

Five-loop reflection. BUILD: repair the baseline. ASK WHY: compare item, profile, raw overlap, and collaborative alternatives. TEST: run unit/data checks and one transparent cosine derivation. EXPLAIN: distinguish relevance, feature-count bias, and discovery. IMPROVE: propose a profile-item blend with novelty guardrails.

References

- [1] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, Art. 19, pp. 1-19, Dec. 2015. doi: 10.1145/2827872. [Online]. Available: <https://doi.org/10.1145/2827872>. [Accessed: 2026-09-22].
- [2] GroupLens Research, "MovieLens 100K Dataset README," University of Minnesota, 1998. [Online]. Available: <https://files.grouplens.org/datasets/movielens/ml-100k-README.txt>. [Accessed: 2026-09-22].
- [3] M. J. Pazzani and D. Billsus, "Content-Based Recommendation Systems," in *The Adaptive Web*, LNCS 4321. Berlin, Germany: Springer, 2007, pp. 325-341. doi: 10.1007/978-3-540-72079-9_10. [Online]. Available: https://doi.org/10.1007/978-3-540-72079-9_10. [Accessed: 2026-09-22].
- [4] P. Lops, M. de Gemmis, and G. Semeraro, "Content-based Recommender Systems: State of the Art and Trends," in *Recommender Systems Handbook*. Boston, MA, USA: Springer, 2011, pp. 73-105. doi: 10.1007/978-0-387-85820-3_3. [Online]. Available: https://doi.org/10.1007/978-0-387-85820-3_3. [Accessed: 2026-09-22].
- [5] WHATWG, "Fetch Standard," Living Standard. [Online]. Available: <https://fetch.spec.whatwg.org/>. [Accessed: 2026-09-22].

Reproducibility note. All reported recommendation metrics were regenerated from the checked-in u.item and u.data files with the same 19-dimensional representation, Bayesian tie-breaker, and candidate exclusions used by the deployed application. References [1]-[5] were checked against the publisher, dataset owner, or standards body pages before submission.

Submission companion. The required session.json must be exported directly from the AI coding tool and submitted without modification. It is not embedded in this PDF.